

บทที่ 2

การสุ่มเลขเทียม การแปลงข้อมูลและเข้ารหัสลับ

ในการทำโปรเจกต์นี้ต้องอาศัยความรู้จากสามส่วนด้วยกันคือส่วนของทฤษฎีการเข้ารหัสและถอดรหัสลับ ซึ่งก็มีสามส่วนคือส่วนเลขสุ่มเทียม ส่วนของการเข้ารหัสทางเดียว และส่วนของเข้ารหัสและถอดรหัสลับ ทำการศึกษาว่าทฤษฎีนี้มีข้อดีและข้อเสียอย่างไรเพื่อหาว่าทฤษฎีไหนที่เหมาะสมกับโปรเจกต์ของเรา ถัดจากส่วนของการเข้ารหัสและถอดรหัสก็มีส่วนของซอฟต์แวร์ดีไวซ์ใดเวอร์ซึ่งเราต้องเขียนขึ้นมาใหม่ให้สามารถใช้กับฮาร์ดแวร์ที่เราสร้างขึ้นมาเอง ต้องมีการศึกษาว่าจะติดต่อกับฮาร์ดแวร์อย่างไร ไม่ว่าจะเป็นเรื่องของการรับส่งข้อมูลระหว่างฮาร์ดแวร์กับซอฟต์แวร์แอปพลิเคชัน และสุดท้ายส่วนของฮาร์ดแวร์เป็นส่วนศึกษาการเขียนภาษาเอชดีเอลเพื่อให้สามารถทำให้การ์ดทำงานได้และยังต้องศึกษาการทำงานของพีซีไอด้วย

2.1 ทฤษฎีทางด้านการสุ่มเลขเทียม

การสุ่มเลขนั้นมีประโยชน์ในการนำมาเพิ่มความปลอดภัยให้กับคีย์ที่เราใช้ในการเข้ารหัสลับ ซึ่งโดยปกติแล้วเป็นอัลกอริธึมที่นำมาใช้เพื่อเพิ่มประสิทธิภาพด้านความปลอดภัยให้กับส่วนต่างๆ เช่น คีย์ที่ใช้ในการเข้ารหัสลับของเรา ค่าเวกเตอร์ตั้งต้น หมายเลขเซสชัน (session id) โดยจะเป็นการเพิ่มความยากให้กับผู้ที่ไม่หวังดี (ที่อาจอยากเปลี่ยนแปลงข้อมูลหรือขโมยข้อมูล) ในการคาดเดาค่าต่างๆ โดยการสุ่มตัวเลข (random number) นั้นมีทั้งแบบการสุ่มเลขแบบแท้จริง (truly-random number) และการสุ่มเลขเทียม (pseudo random number) โดยการสุ่มเลขแบบแท้นั้นเป็นการสุ่มที่ไม่มีแบบแผนใด ๆ ที่จะทำให้สามารถคาดเดาเลขต่อไปได้ว่าจะเป็นค่าอะไร ซึ่งทำให้เราก็ไม่อาจแน่ใจได้ว่าเลขที่ได้ออกมาจากการสุ่มนั้นจะเป็นค่าการสุ่มตัวเลขที่เหมาะสมหรือเปล่า (เลขที่ออกมาอาจมีค่าซ้ำกัน) โดยปกติทั่วไปแล้วการสุ่มอย่างแท้นั้นจะเป็นการเอาค่ามาจากส่วนของฮาร์ดแวร์ ตัวอย่างเช่น การสุ่มเลขที่ได้จากเวลาของจังหวะในการพิมพ์ (key stroke) ค่าจากหมายเลขของกระบวนการที่กำลังทำงานอยู่ (running process id) ค่าจากนาฬิกา โดยค่าที่ได้มาเหล่านี้เราก็ไม่อาจแน่ใจได้อยู่ดี ซึ่งเป็นการยากมากที่จะสามารถหาแหล่งที่จะให้ค่าที่เป็นการสุ่มอย่างแท้จริงตามที่ได้กล่าวมาแล้ว ซึ่งบางทีการเลือกค่าตั้งต้นในการสุ่มเลขเทียมนั้นจะได้จากค่าการสุ่มเลขจริง

คุณสมบัติของการสุ่มเลขแบบแท้นั้นคือ

1. ต้องไม่มีความโน้มเอียง คือตัวเลขทุกตัวมีความเป็นไปได้ในการเป็นผลลัพธ์ดังต่อไปนี้

2. ต้องไม่สามารถคาดเดาได้ คือ ไม่สามารถคาดเดาผลลัพธ์ตัวต่อไปได้ว่าจะเป็นค่าใดแม้ว่าจะรู้ผลลัพธ์ที่ผ่านมาก็ไม่สามารถรู้ค่าต่อไปได้
3. ต้องไม่สามารถให้ผลลัพธ์ในรูปแบบเดียวกันคือหากว่าเรามีการสุ่มตัวเลขสองชุด โดยใช้เงื่อนไขในการเริ่มต้นที่เหมือนกัน ก็จะได้ค่าตัวเลขที่ออกมาต่างกัน

เนื่องจากข้อจำกัดที่ว่า การสุ่มอย่างแท้จริงนั้น ไม่อาจแน่ใจได้ว่าเป็นค่าที่ได้ออกมาเป็นค่าที่ควรได้จากการสุ่มที่สุ่มหรือเปล่า ดังนั้นจึงมีการใช้การสุ่มเลขเทียม (pseudo random number) เข้ามาแทน โดยการสุ่มเลขเทียมนั้นเป็นอัลกอริทึมที่สามารถรู้ค่าต่อไปได้ (deterministic algorithm) ซึ่งแน่นอนว่าเลขที่ได้ออกมานั้นเป็นการใช้การคำนวณค่าทางคณิตศาสตร์ออกมา

คุณสมบัติของการสุ่มเลขเทียมคือ

1. จะต้องมียุคเริ่มต้น (seed) เป็นอินพุตในการทำงานถึงจะให้ค่าผลลัพธ์ออกมา
2. ลำดับของตัวเลขที่ได้ออกมานั้นเป็นผลต่อเนื่องที่ได้จากค่าตั้งต้น ดังนั้นถ้าเราใช้ค่าตั้งต้นเป็นค่าเดิม ลำดับของเลขที่เป็นผลลัพธ์จะต้องเป็นค่าเดิมอยู่ในลำดับเดิม
3. เนื่องจากเป็นผลลัพธ์ที่ได้มาจากการคำนวณดังนั้นย่อมรู้ได้ว่าค่าที่ออกมาตัวต่อไปจะเป็นค่าใด

ค่าตั้งต้น(seed) คือ ค่าที่เป็นเหมือนอินพุตให้กับการทำงาน ซึ่งเราจะกำหนดค่านี้ในครั้งแรกที่เริ่มใช้งานเท่านั้นหลังจากนั้นก็จะได้ผลลัพธ์เป็นตัวเลขออกมาเรื่อย ๆ จนกว่าการสุ่มเลขออกมานั้นจะครบจำนวนเท่าที่อัลกอริทึมนี้รองรับได้ หรือเมื่อต้องการให้มีการกำหนดค่าเริ่มต้นไปใหม่

ลักษณะของการสุ่มเลขเทียมที่ดีคือ

1. ถ้าเราพิจารณาจากลำดับของตัวเลขที่เป็นผลลัพธ์ออกมานั้น จะต้องไม่สามารถหาความสัมพันธ์ของผลลัพธ์นั้นเพื่อที่จะเอาไปใช้ในการคาดเดาตัวเลขที่เป็นผลลัพธ์ตัวต่อไปได้
2. ตัวเลขซึ่งคือผลลัพธ์ที่ออกมานั้นจะต้องมีค่าไม่ซ้ำกันในช่วงที่กำหนดซึ่งขึ้นอยู่กับอัลกอริทึมที่เลือกใช้
3. ช่วงความยาวที่ให้ค่าไม่ซ้ำกันจะต้องมีความยาวมากพอสมควรเพื่อให้ค่าที่ออกมาไม่ต้องการเริ่มใส่ค่าตั้งต้นใหม่ ไม่ต้องกลัวว่าจะเกิดค่าที่ซ้ำกัน หากว่าช่วงความยาวของรอบสั้นก็ต้องเริ่มใหม่อาจให้ค่าที่ซ้ำ ๆ เดิมออกมา ตัวอย่างลักษณะที่ไม่ดีที่จะทำให้ข้อผิดพลาดในการทำเลขสุ่มเทียม

ตัวอย่างลักษณะที่ไม่ดีที่จะทำให้ข้อผิดพลาดในการทำเลขสุ่มเทียม

1. การเลือกใช้ค่าตั้งต้นที่มีขนาดเล็กไป(จำนวนบิตน้อยไป) เช่นการเลือกค่าตั้งต้นที่มีขนาด 16 บิตเพื่อนำมาใช้เป็นคีย์ในการเข้ารหัสลับตามอัลกอริทึมที่เลือก ปัญหาคือ คีย์ที่เป็นไปได้เพียงแค่ 65536 ค่าเท่านั้นที่สามารถเป็นไปได้ ซึ่งเป็นค่าที่ถือว่าน้อย

ในการที่จะค้นหาค่าที่เป็นไปได้(เนื่องจากค่าที่เป็นไปได้นั้นมีจำนวนน้อยก็หมายความว่าสามารถหาค่าที่เป็นไปได้ในช่วงเวลาที่สั้นลง)

2. การเลือกค่าตั้งต้นโดยใช้ค่าจากนาฬิกา ซึ่งหากพิจารณาว่าหน่วยที่ใช้คือหน่วยของวินาที ดังนั้นค่าที่เป็นไปได้อาจจะมีทั้งหมด $60 \times 60 \times 60$ เท่ากับ 216000 ค่าที่เป็นไปได้ซึ่งเป็นจำนวนที่ไม่มากนัก
3. การเลือกค่าตั้งต้นที่มีความยาวน้อยไปย่อมส่งผลให้มีจำนวนค่าที่มีความเป็นไปได้น้อยลง สามารถคำนวณหรือคาดเดาได้ง่ายขึ้น

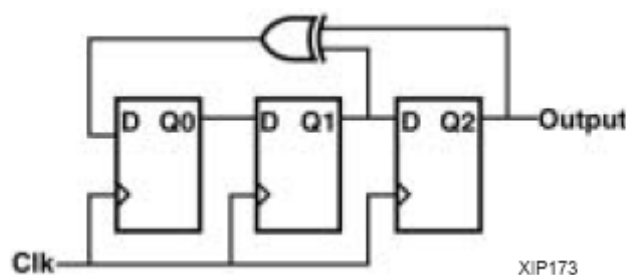
อัลกอริทึมสำหรับการสุ่มเทียมนี้มีอยู่มากมายหลายอัลกอริทึมเช่น linear congruential generators, lagged Fibonacci generators, Linear feedback shift registers, generalised feedback shift registers, Blum Blum Shub, Mersenne twister, Fortuna

2.2 อัลกอริทึม Linear feedback shift registers

อัลกอริทึมแบบสุ่มเลขเทียมที่เป็นซิงโครไนซ์โดยบิตที่เป็นค่าอินพุตนั้นเป็นค่าที่ถูกบังคับจากฟังก์ชันลิเนียร์ (linear function) โดยฟังก์ชันลิเนียร์ส่วนใหญ่ที่ใช้ในการทำงานของอัลกอริทึมนี้จะเป็น XOR หรือ XNOR จากตำแหน่งเก็บของค่าเริ่มต้น ค่าเริ่มต้นที่ใช้จะเรียกว่าค่าตั้งต้น(seed) ซึ่งยกเว้นไม่ควรใช้ค่าตั้งต้นเป็น 0 เนื่องจากจะทำให้ค่าผลลัพธ์ออกมาเป็น 0 ทำให้สามารถคาดเดาได้ อัลกอริทึม LFSR นี้สามารถทำงานได้อย่างรวดเร็ว

อัลกอริทึมที่เลือกใช้ในโครงการได้ทำเป็น LFSR ขนาด 128 บิต LFSR สามารถแบ่งออกได้เป็น 2 แบบคือแบบ Fibonacci และแบบ Galois โดยแบบ

1. Fibonacci ลักษณะการทำงานคือ ค่าอินพุตจะได้รับการเอ็กซ์คลูซีฟพอร์ค่าเอาต์พุตที่ผัดแบบกลับไป ดังตัวอย่างข้างล่างเป็นรูปการทำงานของ Fibonacci LFSR ขนาด 3 บิต



รูปที่ 2.1 แสดง LFSR แบบ Fibonacci ขนาด 3 บิต

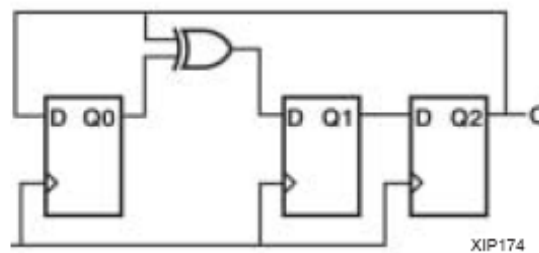
ซึ่งขั้นตอนการทำงานนั้น จะทำการโหลดค่าตั้งต้น (seed) ซึ่งข้อยกเว้นคือห้ามเป็นค่าศูนย์ ตัวอย่างของผลลัพธ์ที่ได้ออกมาเป็นค่า Q2 จากตาราง ตัวอย่างผลลัพธ์เป็นดังตารางนี้

Q2	Q1	Q0
1	1	1
1	1	0
1	0	0
0	0	1
0	1	0
1	0	1
0	1	1
1	1	1

รูปที่ 2.2 แสดงค่าผลลัพธ์ที่ได้จาก LFSR แบบ Fibonacci ขนาด 3 บิต

ซึ่งลำดับการสุ่มตัวเลขที่ได้ออกมานั้นสามารถใช้เอ็กซ์คลูซีฟนอร์ได้ด้วยซึ่งในกรณีนี้ค่าตั้งต้น (seed) ไม่ควรจะเป็น 1 ทุกบิต

2. Galois ลักษณะการทำงานเป็นดังรูปข้างล่าง ซึ่งเป็น LFSR ขนาด 3 บิตเช่นกัน



รูปที่ 2.3 แสดง LFSR แบบ Galois ขนาด 3 บิต

ตารางข้างล่างนี้แสดงค่าเอาต์พุตที่ได้จากการชีพ 1 รอบ

Q2	Q1	Q0
1	1	1
1	0	1
0	0	1
0	1	0
1	0	0
0	1	1
1	1	0
1	1	1

รูปที่ 2.4 แสดงค่าผลลัพธ์ที่ได้จาก LFSR แบบ Galois ขนาด 3 บิต

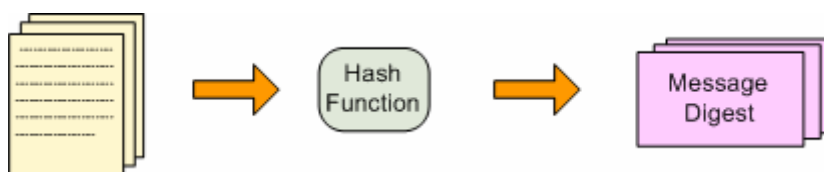
ค่าที่เก็บคือ ตำแหน่งที่จะส่งค่ากลับคืนเพื่อนำมาใช้ในการทำเอ็กซ์คลูซีฟอออร์(XOR) เป็นค่าที่เอามาใช้เป็นค่าอินพุต ซึ่งตำแหน่งของค่าที่เก็บนั้นจะมีความแตกต่างกันขึ้นอยู่กับจำนวนของรีจิสเตอร์ที่ต้องการ ตารางที่จะแสดงในภาคผนวกจะแสดงค่าที่เก็บเพื่อสร้างเอาต์พุตออกมาให้มีช่วงมากที่สุดซึ่งดูค่าที่เก็บได้จากตารางในภาคผนวก ก. นี้จะแสดงค่าที่เก็บเพื่อสร้างออกมาให้มีลำดับค่ามากที่สุด

ขนาดของ LFSR ที่เป็นไปได้ คือตั้งแต่ขนาด 2-168 บิต

ช่วงของผลลัพธ์ที่มากที่สุดที่ได้ออกมาโดยใช้อัลกอริทึม LFSR นั้นจะมีค่าเป็น $2^n - 1$ โดยค่า n นั้นเป็นจำนวนของรีจิสเตอร์ที่ใช้ ดังนั้นหากใช้รีจิสเตอร์ขนาด 32 บิต จะให้ค่าผลลัพธ์ออกมาถึง 4 ล้านค่า($2^{32} - 1$)

2.3 การแปลงข้อมูล

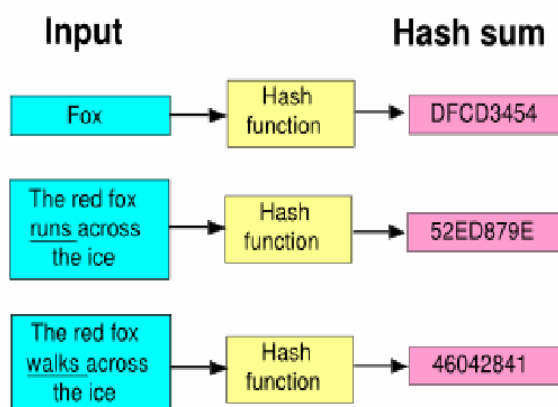
เมสเชสไดเจสต์เป็นวิธีที่สำคัญที่สามารถใช้ในการตรวจสอบว่าไฟล์ในระบบที่ใช้งานมีการเปลี่ยนแปลงแก้ไขหรือไม่ (ไม่ว่าจะโดยเจตนาหรือไม่ก็ตาม) บางครั้งการเปลี่ยนแปลงแก้ไขอาจถูกกระทำโดยผู้ที่ไม่มีความรู้ เช่น ผู้บุกรุก เป็นต้น วิธีการใช้เมสเชสไดเจสต์เพื่อตรวจสอบไฟล์ในระบบหรือการส่งข้อมูลหากันผ่านทางเครือข่าย คือให้เลือกใช้อัลกอริทึมใดอัลกอริทึมหนึ่ง เช่น MD5 เพื่อสร้างเมสเชสไดเจสต์ของไฟล์ที่จะส่งหากันแล้วทำการส่งไป เมื่อผู้รับได้รับข้อความก็จะทำการหาเมสเชสไดเจสต์แล้วเอาค่าที่ได้มาเปรียบเทียบกับค่าเมสเชสไดเจสต์ที่ส่งมาถ้าได้เหมือนกันก็จะสามารถมั่นใจได้ว่าข้อความไม่ได้ถูกเปลี่ยนแปลง แต่หากไม่เท่ากันก็แสดงว่าข้อความถูกเปลี่ยนแปลง นอกจากนี้เมสเชสไดเจสต์ยังเป็นส่วนหนึ่งของการลงลายมือชื่ออิเล็กทรอนิกส์ กล่าวคือการลงลายมือชื่ออิเล็กทรอนิกส์ในปัจจุบันจะทำการลงลายมือชื่อกับเมสเชสไดเจสต์ของข้อความตั้งต้นแทนการลงลายมือชื่อกับข้อความตั้งต้นทั้งข้อความ



รูปที่ 2.5 แสดงตัวอย่างการทำแฮชฟังก์ชัน

ลักษณะของอัลกอริทึมในการทำเมสเชสไดเจสต์คือ การสร้างข้อความสรุปที่สามารถใช้เป็นตัวแทนของข้อความตั้งต้นได้ โดยทั่วไปเมสเชสไดเจสต์ที่ได้ออกมาจะมีความยาวตั้งแต่ 128 ขึ้นไปขึ้นอยู่กับอัลกอริทึมที่เลือกใช้และจะไม่ขึ้นกับขนาดความยาวของข้อความตั้งต้น ในการทำแฮช (hash) หรือ เมสเชสไดเจสต์นั้นเป็นการทำฟังก์ชันทางเดียว ซึ่งลักษณะของอัลกอริทึมที่ดีคือ

1. ถ้าเปลี่ยนแปลงอักขรไปแม้แต่บิตเดียว ต้องให้ได้ผลลัพธ์ที่ไม่เหมือนเดิมเลย ไม่งั้นคนจะจับรูปแบบการทำงานได้ เช่น $f(x) = 12345$, $f(x+1) = 99999$ เป็นต้น ทำให้หาความสัมพันธ์ได้ยาก ทุกๆ บิตของไคเจสต์จะมีโอกาสร้อยละ 50 ที่จะแปรเปลี่ยนค่าไปด้วย ซึ่งหมายถึงว่า 0 เปลี่ยนค่าเป็น 1 และ 1 เปลี่ยนเป็น 0 คุณสมบัติข้อนี้สามารถอธิบายได้ว่าการเปลี่ยนแปลงแก้ไขข้อความตั้งต้นโดยผู้ไม่ประสงค์ดีแม้ว่าอาจแก้ไขเพียงเล็กน้อยก็ตาม ก็จะส่งผลให้ผู้รับข้อความทราบว่าข้อความที่ตนได้รับไม่ใช่ข้อความตั้งต้น (โดยการนำข้อความที่ตนได้รับเข้าอัลกอริทึมเพื่อทำการคำนวณหาไคเจสต์ออกมา แล้วจึงเปรียบเทียบไคเจสต์ที่คำนวณได้กับไคเจสต์ที่ส่งมาให้ด้วย ถ้าต่างกัน แสดงว่าข้อความที่ได้นั้นถูกเปลี่ยนแปลงแก้ไข)



รูปที่ 2.6 ตัวอย่างการทำงานของแฮช

2. ต้องไม่ใช่ฟังก์ชันแบบ 1:1 เพราะสามารถทำ mapping ได้ง่ายฟังก์ชันที่ใช้ส่วนใหญ่จะเป็น modulo เพราะ มีคุณสมบัติที่ดี $X \bmod 5 = 3$ ตัวเลขอะไรข้างที่หาร 5 แล้วเหลือเศษ 3 คำตอบมีได้เป็นล้าน ๆ คำตอบเลย

3. มีการชนกัน (collision) ที่ต่ำ การชนกันหมายถึงการที่มีข้อความตั้งต้นสองข้อความที่แตกต่างกันแต่เมื่อคำนวณเมสเชสไคเจสต์แล้วได้ออกมาค่าเดียวกัน อย่างตัวอย่างข้อ 2 เกิดการชนกันที่สูงมากทุก ๆ 5 ตัวจะได้คำตอบที่ใช้ได้ ถึงจะไม่ใช้คำตอบ แต่ผลลัพธ์ดันเท่ากัน ก็ถือว่าถูกอย่างนี้ไม่ควรใช้ โอกาสที่ข้อความตั้งต้น 2 ข้อความใดๆ ที่มีความแตกต่างกัน จะสามารถคำนวณได้ค่าไคเจสต์เดียวกันมีโอกาสน้อยมาก คุณสมบัติข้อนี้ทำให้แน่ใจได้ว่า เมื่อผู้ไม่ประสงค์ดีทำการแก้ไขข้อความตั้งต้น ผู้รับข้อความที่ถูกแก้ไขไปแล้วนั้นจะสามารถตรวจพบได้ถึงความผิดปกติที่เกิดขึ้นอย่างแน่นอน

อย่างไรก็ตามในทางทฤษฎีแล้ว มีโอกาสที่ข้อความ 2 ข้อความที่แตกต่างกันจะสามารถคำนวณแล้วได้ค่าไคเจสต์เดียวกัน แต่ก็มีข้อแม้ในเรื่องของเวลานั่นเองที่ทำให้สามารถใช้วิธีนี้ได้ อัลกอริทึมสำหรับสร้างไคเจสต์ที่ดีควรมีโอกาสน้อยมากๆ ที่จะก่อให้เกิดปัญหาการชนกันของไคเจสต์

โดยปัญหานี้ทำให้วิธีการตรวจสอบความมั่นคงปลอดภัยของข้อมูลที่อาศัยความเป็นหนึ่งเดียวของเมสเสจไคเจสต์ว่าไม่น่าเชื่อถือ เช่น ทำให้ไม่สามารถยืนยันความครบถ้วนแท้จริงของซอฟต์แวร์สำหรับดาวน์โหลดด้วยเมสเสจไคเจสต์ของซอฟต์แวร์นั้นได้ หรือ เป็นการเพิ่มโอกาสให้บุคคลที่ถือโงสามารถเขียน E-mail เพื่อจะแจ้งความจำนงค์ในการขอปิดบัญชีของเจ้าของบัญชีได้โดยใช้ลายมือชื่อดิจิทัลปลอม โดยเมสเสจไคเจสต์ที่ทำการออกมานั้นเป็นตัวตรวจสอบความถูกต้องของข้อความว่าข้อความที่ส่งไปนั้นมีการเปลี่ยนแปลงไปหรือไม่

อัลกอริทึมสำหรับสร้างไคเจสต์ที่เป็นที่รู้จักกันดีมีดังนี้

อัลกอริทึม MD2

ผู้พัฒนาคือ Ronald Rivest ข้อเสียของอัลกอริทึมนี้คือใช้เวลามากในการคำนวณไคเจสต์ต่างๆ MD2 จึงไม่ค่อยได้มีการใช้งานกันมากนัก MD2 สร้างไคเจสต์ที่มีความยาว 128 บิต

อัลกอริทึม MD4

อัลกอริทึมนี้พัฒนาขึ้นมาเพื่อแก้ปัญหาความล่าช้าในการคำนวณของ MD2 อย่างไรก็ตามในภายหลังได้พบว่าอัลกอริทึมมีข้อบกพร่อง คือ มีปัญหาการชนกันของไคเจสต์มีโอกาสเกิดขึ้นได้ไม่น้อย ซึ่งผู้บุกรุกอาจใช้ประโยชน์จากจุดอ่อนนี้เพื่อทำการแก้ไขข้อความตั้งต้นที่ส่งมาให้ได้ MD4 ผลิตไคเจสต์ที่มีขนาด 128 บิต

อัลกอริทึม MD5

พัฒนาต่อจาก MD4 เพื่อให้มีความปลอดภัยที่สูงขึ้น ถึงแม้จะเป็นที่นิยมใช้งานกันอย่างแพร่หลาย แต่ก็ยังมีจุดบกพร่องของ MD5 (เช่นเดียวกับ MD4) จึงทำให้ความนิยมเริ่มลดลง MD5 ผลิตไคเจสต์ที่มีขนาด 128 บิต

อัลกอริทึม SHA

SHA ย่อจาก Secure Hash Algorithm อัลกอริทึม SHA ได้รับแนวคิดในการพัฒนามาจาก MD4 และได้รับการพัฒนาขึ้นมาเพื่อใช้งานร่วมกับอัลกอริทึม DSS (ซึ่งใช้ในการลงลายมือชื่ออิเล็กทรอนิกส์) หลังจากที่ได้มีการตีพิมพ์เผยแพร่อัลกอริทึมนี้ได้ไม่นาน NIST ก็ประกาศตามมาว่าอัลกอริทึมจำเป็นต้องได้รับการแก้ไขเพิ่มเติมเล็กน้อยเพื่อให้สามารถใช้งานได้เหมาะสม SHA สร้างไคเจสต์ที่มีขนาด 160 บิต

อัลกอริทึม SHA-1

SHA-1 เป็นอัลกอริทึมที่แก้ไขเพิ่มเติมเล็กน้อยจาก SHA การแก้ไขเพิ่มเติมนี้เป็นที่เชื่อกันว่าทำให้อัลกอริทึม SHA-1 มีความปลอดภัยที่สูงขึ้น SHA-1 สร้างไคเจสต์ที่มีขนาด 160 บิต

อัลกอริทึม SHA-256, SHA-384 และ SHA-512

เป็นอัลกอริทึมที่แก้ไขขนาดให้มีความยาวมากขึ้น ทำให้มีความปลอดภัยมากขึ้นกว่าเดิม เนื่องจากขนาดของเมสเสจไคเจสต์ที่ออกมา มีขนาดที่ยาวขึ้นก็ต้องใช้เวลามากขึ้น

2.4 การแปลงข้อมูลโดยใช้การคำนวณแบบ SHA1

การทำงานของ SHA1 มีขั้นตอนดังนี้

ข้อจำกัดของ SHA1 คือ ข้อความที่เป็นอินพุตเข้ามาจะต้องมีขนาดน้อยกว่า 2^{64} และเมสเชสไคเจสต์ที่ได้ออกมาจะมีขนาด 160 บิต โดยค่าที่ได้ออกมาเป็นการคำนวณค่าถึง 80 รอบ แล้วได้ผลลัพธ์ออกมาเป็น A B C D E ซึ่งแต่ละตัวมีค่าตัวละ 32 บิต 5 ตัวมาต่อกันจะได้เป็นเมสเชสไคเจสต์ที่มีขนาด 160 บิต

ขั้นตอนการทำงานคือ

1. การเติมเต็มบิตต่อท้าย(Padding Bit)

ทำการแบ่งข้อความที่รับเข้ามาเป็นบล็อกๆ โดยมีขนาดบล็อกละ 512 บิต และในบล็อกสุดท้ายจะทำการเติมจำนวนบิตให้ครบ 448 บิต และอีก 64 บิตที่เหลือใช้ในการเติมความยาวของข้อความ และการเติมบิตต่อท้ายนั้นเริ่มต้นด้วยการเติมบิต “1” ก่อน แล้วจึงตามด้วยบิต “0” ต่อท้ายไปจนกว่าความยาวของบิตข้อความในบล็อกนั้นจะครบ 448 บิต ขั้นตอนดังตัวอย่างข้างล่างนี้

สมมติให้ข้อความคือ ABC ก็จะมีเป็นแอสกีได้เป็น 616263

แปลงเป็นเลขฐานสองได้ 01100001 01100010 01100011

- เติม 1 หลังข้อความนั้น

01100001 01100010 01100011 1

- เติม 0 ให้มีขนาด 448 บิต(เติม 0 ไปอีก 423 บิตโดยแสดงเป็นเลขฐาน16)

61626380 00000000 00000000 00000000 00000000

00000000 00000000 00000000 00000000 00000000

00000000 00000000 00000000 00000000

2. การเติมความยาวบิตของข้อความ

ค่าความยาวของข้อความที่เป็นอินพุตนั้นจะเก็บไว้ใน 64 บิตที่เหลือไว้ เนื่องจากเป็นข้อกำหนดว่าความยาวของอินพุตที่สามารถใช้อัลกอริธึม SHA1 นั้นมีขนาดไม่เกิน 2^{64} บิต ดังนั้นพื้นที่ 64 บิตที่เหลือไว้จึงเพียงพอที่จะใช้ในการเก็บค่าความยาวบิตของข้อความ โดยเติมเวิร์ด(word) 32 บิตที่มีลำดับต่ำก่อนแล้วจึงตามด้วยเวิร์ดที่มีลำดับสูง ดังนั้นข้อความที่ได้ออกมาจะมีขนาดเป็นหลายเท่าของ 512 บิต หรือมีขนาดเป็นหลายเท่าของ 16 เวิร์ด (1 เวิร์ด มีขนาด 32 บิต) ให้ $M[0.....N-1]$ แทรกเวิร์ดของข้อความที่ผ่านกระบวนการต่อเติมแล้ว โดยที่ N คือ จำนวนเท่าของ 16 ตัวอย่างเช่น

- เติม ความยาวของข้อความนั้นไปในส่วนท้ายสุดของบล็อกนั้น
ความยาวของข้อความคือ 24 ดังนั้นเติมไปในส่วนท้ายของบล็อกนี้ (แสดงเป็น
เลขฐานสิบหก)

61626380 00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000
00000018

3. การกำหนดค่าเริ่มต้นของบัพเฟอร์ของเมสเชสไคเจสต์

บัพเฟอร์มีขนาด 160 บิต เพื่อเก็บผลลัพธ์ของการแฮช(Hash) โดยจะใช้บัพเฟอร์ที่มีขนาด 32 บิตทั้งหมด 5 ตัวคือ A B C D และ E โดยบัพเฟอร์เหล่านี้มีค่าเริ่มต้นในรูปของเลขฐาน 16 ดังนี้ ซึ่งเรียงลำดับจากไบต์ที่มีลำดับต่ำสุดก่อน

4. กระบวนการหาเมสเชสไคเจสต์

ฟังก์ชันที่ใช้คือ $f_0, f_1, f_2, \dots, f_{79}$ ที่ใช้ในการคำนวณ 80 รอบ

คือสำหรับ $f_t, 0 \leq t \leq 79$ โดยมีการกำหนดค่าดังนี้

$$F_t(B,C,D) = (B \text{ AND } C) \text{ OR } ((\text{NOT } B) \text{ AND } D) \quad ; (0 \leq t \leq 19)$$

$$F_t(B,C,D) = B \text{ XOR } C \text{ XOR } D \quad ; (20 \leq t \leq 39)$$

$$F_t(B,C,D) = (B \text{ AND } C) \text{ OR } (B \text{ AND } D) \text{ OR } (C \text{ AND } D) \quad ; (40 \leq t \leq 59)$$

$$F_t(B,C,D) = B \text{ XOR } C \text{ XOR } D \quad ; (60 \leq t \leq 79)$$

โอเปอร์เรชันที่ใช้

เนื่องจากเป็นพจน์หนึ่งที่ใช้ในการคำนวณคือ

$$S^n(X) = (X \ll n) \text{ OR } (X \gg 32-n)$$

ค่าคงที่ที่ใช้ในการคำนวณ คือ

$$K_t = 5A827999 \quad (0 \leq t \leq 19)$$

$$K_t = 6ED9EBA1 \quad (20 \leq t \leq 39)$$

$$K_t = 8F1BBCDC \quad (40 \leq t \leq 59)$$

$$K_t = CA62C1D6 \quad (60 \leq t \leq 79)$$

การคำนวณจะมีการคำนวณเพื่อให้ได้ Temp ออกมา

โดยมีการกำหนดค่าเริ่มต้นในการคำนวณ คือ $\{H_t\}$ เพื่อใช้ในการคำนวณดังนี้ (แสดงใน
รูปแบบเลขฐาน 16)

$$H_0 = 67452301$$

$$H_1 = \text{EFCDAB89}$$

$$H_2 = 98BADCFE$$

$$H_3 = 10325476$$

$$H_4 = \text{C3D2E1F0}$$

- แบ่งบล็อกออกเป็นเวิร์ด(word) เวิร์ดหนึ่งมี 32 บิตจะได้ทั้งหมด 16 เวิร์ดคือแทนด้วย $W[0]$ ถึง $W[15]$ คือ

$$W[0] = 61626380 \quad W[1] = 00000000 \quad W[2] = 00000000$$

$$W[3] = 00000000 \quad W[4] = 00000000 \quad W[5] = 00000000$$

$$W[6] = 00000000 \quad W[7] = 00000000 \quad W[8] = 00000000$$

$$W[9] = 00000000 \quad W[10] = 00000000 \quad W[11] = 00000000$$

$$W[12] = 00000000 \quad W[13] = 00000000 \quad W[14] = 00000000$$

$$W[15] = 00000000$$

และสำหรับค่า $t = 16$ ถึง 79 จะหาค่า W_t ได้จาก $W_t = S1(W_{t-3} \text{ XOR } W_{t-8} \text{ XOR } W_{t-14} \text{ XOR } W_{t-16})$

- จากนั้นกำหนดให้ $A = H_0$, $B = H_1$, $C = H_2$, $D = H_3$, $E = H_4$
- จากนั้นทำการคำนวณโดยกำหนดว่าจาก $t=0$ ถึง 79 ให้ทำ
 - $\text{TEMP} = S^5(A) + F_t(B,C,D) + E + W_t + K_t$
 - กำหนดให้ $E = D$; $D = C$; $C = S^{30}(B)$; $B = A$; $A = \text{TEMP}$
- เมื่อคำนวณครบ 80 รอบจะได้

$$H_0 = H_0 + A, H_1 = H_1 + B, H_2 = H_2 + C, H_3 = H_3 + D, H_4 = H_4 + E$$

ก็จะได้เมสเสจไคเจสต์ขนาด 160 บิตที่ได้จากการเรียงกัน 5 words คือ $H_0 H_1 H_2 H_3 H_4$

$H_3 H_4$

หมายเหตุ หากข้อความที่จะทำเมสเสจไคเจสต์นั้นมีความยาวเกิน 512 บิต ค่า $H_0 H_1 H_2 H_3 H_4$ ที่ได้จากบล็อกแรกก็จะไปใช้เป็นค่า $H_0 H_1 H_2 H_3 H_4$ ในบล็อกต่อไป

2.5 การเข้ารหัสลับ

การเข้ารหัสลับข้อมูล คือ การทำให้ข้อมูลเป็นความลับ โดยอาศัยการทำการเข้ารหัสลับ และการถอดรหัสลับซึ่งเป็นส่วนสำคัญของระบบข้อมูลในปัจจุบันนี้ จึงมองเป็น 2 ส่วนหลักๆคือ เรื่องการเข้ารหัสลับ และการถอดรหัสลับ

การเข้ารหัสลับ(Encryption) เป็นการเปลี่ยนรูปข้อมูลโดยใช้กระบวนการต่างๆในการเปลี่ยนรูปข้อมูลเพื่อให้ไม่สามารถอ่านหรือตีค่าแปลความหมายได้ เพื่อทำให้ข้อมูลนั้นเป็นความลับโดยวิธีในการเข้ารหัสนั้นโดยพื้นฐานมีหลักการที่สำคัญ 2 อย่างคือ

1. การแทนที่ (Substitution) เป็นการแทนที่บิตนั้นๆด้วยข้อมูลอื่น ทำให้ข้อมูลมีความสับสนยากต่อการถอดรหัส (โดยตำแหน่งบิตนั้นๆ ยังคงเรียงกันเหมือนเดิม)

2. การสับเปลี่ยนตำแหน่ง (Permutation) เป็นการสับเปลี่ยนตำแหน่งใดๆของข้อมูล เมื่อมีการสับเปลี่ยนตำแหน่งมากๆ ทำให้ข้อมูลมีความซับซ้อนยากต่อการถอดรหัส

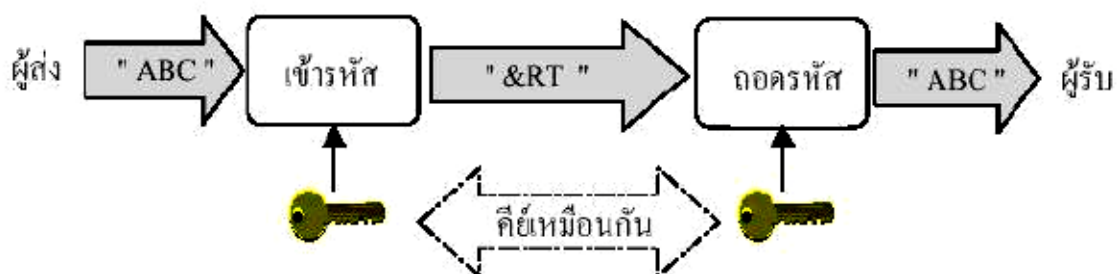
การถอดรหัส (Decryption) เป็นการแปลงข้อมูลที่ได้ผ่านการเข้ารหัสให้กลับมามีอยู่ในสภาพเดิมที่สามารถอ่านและตีความหมายได้ซึ่งการเข้ารหัสและถอดรหัสนั้น ได้ถูกควบคุมโดยกุญแจ (Key) โดยได้แบ่งระบบของการเข้ารหัสตามลักษณะของกุญแจที่ใช้ เป็น 2 แบบ คือ

- ระบบการเข้ารหัสโดยใช้กุญแจเดียว (Single Key Encryption)
- ระบบการเข้ารหัสโดยใช้กุญแจสาธารณะ (Public Key Encryption)

2.5.1 ระบบการเข้ารหัสโดยใช้กุญแจเดียว

อัลกอริทึมแบบนี้จะใช้กุญแจที่เรียกว่า กุญแจลับ (Secret key) ซึ่งมีเพียงหนึ่งเดียวเพื่อใช้ในการเข้ารหัสและถอดรหัสข้อความที่ส่งไป อัลกอริทึมยังสามารถแบ่งย่อยออกเป็น 2 ประเภท ได้แก่ แบบบล็อก (Block Algorithms) ซึ่งจะทำการเข้ารหัสทีละบล็อก (1 บล็อกประกอบด้วยหลายไบต์ เช่น 64 ไบต์ เป็นต้น) และแบบสตรีม (Stream Algorithms) ซึ่งจะทำการเข้ารหัสทีละไบต์อัลกอริทึมแบบนี้จะใช้กุญแจที่เรียกว่า กุญแจลับ (Secret key) ซึ่งมีเพียงหนึ่งเดียวเพื่อใช้ในการเข้ารหัสและถอดรหัสข้อความที่ส่งไป อัลกอริทึมยังสามารถแบ่งย่อยออกเป็น 2 ประเภท ได้แก่ แบบบล็อก (Block Algorithms) ซึ่งจะทำการเข้ารหัสทีละบล็อก (1 บล็อกประกอบด้วยหลายไบต์ เช่น 64 ไบต์ เป็นต้น) และแบบสตรีม (Stream Algorithms) ซึ่งจะทำการเข้ารหัสทีละไบต์

ตัวอย่างของการเข้ารหัสด้วยระบบนี้ ได้แก่ DES, 3DES, IDEA เป็นต้น



รูปที่ 2.7 การเข้ารหัสและถอดรหัสโดยใช้กุญแจเดียว

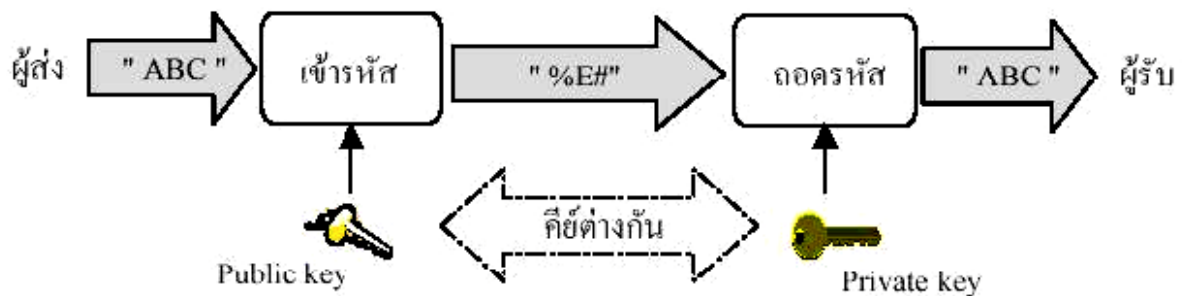
2.5.2 ระบบการเข้ารหัสโดยใช้กุญแจสาธารณะ

การเข้ารหัสโดยใช้กุญแจสาธารณะ ได้นำกุญแจมาประกอบการเข้ารหัส 2 อัน คือ

1. กุญแจส่วนตัว (Private Key) เป็นกุญแจที่จะต้องเก็บเป็นความลับ
2. กุญแจสาธารณะ (Public Key) เป็นกุญแจที่สามารถเปิดเผยให้ผู้อื่นทราบได้

ซึ่งใช้กุญแจที่เรียกกันว่า กุญแจสาธารณะ (Public keys) ในการเข้ารหัสและใช้กุญแจที่เรียกกันว่า กุญแจส่วนตัว (Private keys) ในการถอดรหัสข้อมูลนั้น กุญแจสาธารณะนี้สามารถส่งมอบให้กับผู้อื่นได้ เช่น เพื่อนร่วมงานที่เราต้องการติดต่อด้วย หรือแม้กระทั่งวางไว้บนเว็บไซต์เพื่อให้ผู้อื่นสามารถดาวน์โหลดไปใช้งานได้ สำหรับกุญแจส่วนตัวนั้นต้องเก็บไว้กับผู้เป็นเจ้าของกุญแจส่วนตัวเท่านั้นและห้ามเปิดเผยให้ผู้อื่นทราบโดยเด็ดขาด

ซึ่งกุญแจทั้งสองอันนี้ เป็นกุญแจที่ต่างกัน โดยมีวิธีการเข้าและถอดรหัส ดังรูปนี้



รูปที่ 2.8 การเข้ารหัสและถอดรหัสโดยใช้กุญแจสาธารณะ

2.5.3 ความแข็งแกร่งของอัลกอริทึมสำหรับการเข้ารหัส

ความแข็งแกร่งของอัลกอริทึมหมายถึงความยากในการที่ผู้บุกรุกจะสามารถถอดรหัสข้อมูลได้โดยปราศจากกุญแจที่ใช้ในการเข้ารหัส ซึ่งจะขึ้นอยู่กับปัจจัยดังนี้

- การเก็บกุญแจเข้ารหัสไว้ว่าเป็นความลับ ผู้เป็นเจ้าของกุญแจลับหรือส่วนตัวต้องระมัดระวังไม่ให้กุญแจสูญหายหรือล่วงรู้โดยผู้อื่น
- ความยาวของกุญแจเข้ารหัส ปกติกุญแจเข้ารหัสจะมีความยาวเป็นบิต ยิ่งจำนวนบิตของกุญแจยิ่งมาก ยิ่งทำให้การเดาเพื่อหามุญแจที่ถูกต้องเป็นไปได้ยากยิ่งขึ้น (เช่น กุญแจขนาด 1 บิต จะสามารถแทนตัวเลขได้ 2 ค่าคือ 0 กับ 1 กุญแจขนาด 2 บิต จะเป็นไปได้ 4 ค่าคือ 0, 1, 2, 3 เป็นต้น)
- ความไม่เกรงกลัวต่อการศึกษาหรือคู่อัลกอริทึมเพื่อหารูปแบบของการเข้ารหัส อัลกอริทึมที่ดีต้องเปิดให้ผู้รู้ทำการศึกษาในรายละเอียดได้โดยไม่เกรงว่าผู้ศึกษาจะสามารถจับรูปแบบของการเข้ารหัสได้

- การมีประตูลับในอัลกอริทึม อัลกอริทึมที่ดีต้องไม่เผยไว้ด้วยประตูลับที่สามารถใช้เป็นทางเข้าไปสู่อัลกอริทึม แล้วอาจใช้เพื่อทำการถอดรหัสข้อมูลได้ ประตูลับนี้ทำให้ไม่จำเป็นต้องใช้กุญแจในการถอดรหัส
- ความไม่เกรงกลัวต่อปัญหาการหาความสัมพันธ์ในข้อมูลที่ได้รับ กล่าวคือเมื่อผู้บุกรุกทราบข้อมูลบางอย่างที่เป็นข้อมูลตั้งต้นซึ่งยังไม่ได้เข้ารหัส รวมทั้งมีข้อมูลที่เข้ารหัสแล้ว (ของข้อมูลตั้งต้นนั้น) ผู้บุกรุกอาจจะสามารถหาความสัมพันธ์ระหว่างข้อความทั้งสองนั้นได้ ซึ่งจะเป็นวิธีการในการถอดรหัสข้อมูลได้ ปัญหานี้เรียกกันว่า Known plaintext attack (คำว่า plaintext หมายถึงข้อความตั้งต้นที่ยังไม่ได้ผ่านการเข้ารหัส)
- คุณสมบัติของข้อความตั้งต้น คุณสมบัตินี้อาจใช้เป็นช่องทางในการถอดรหัสข้อมูลได้ อัลกอริทึมที่ดีต้องไม่ใช้คุณสมบัติของข้อความเป็นกลไกในการเข้ารหัสข้อมูล

คำแนะนำในการเลือกใช้อัลกอริทึมคือให้ใช้อัลกอริทึมที่ได้มีการใช้งานมาเป็นระยะเวลานานแล้ว ทั้งนี้เนื่องจากหากปัญหาของอัลกอริทึมนี้มีจริง ก็คงเกิดขึ้นมานานแล้วและก็คงเป็นที่ทราบกันแล้ว นั่นคืออย่างน้อยที่สุดจวบจนกระทั่งถึงปัจจุบัน ก็ยังไม่มีการบุกรุกที่ทำให้อัลกอริทึมนั้นไม่สามารถใช้งานได้อย่างปลอดภัยเป็นที่ประจักษ์ ดังนั้นจึงไม่ควรใช้อัลกอริทึมใหม่ๆ ที่เพิ่งได้มีการนำเสนอกันสู่สาธารณะ เพราะอาจมีช่องโหว่แฝงอยู่และยังไม่เป็นที่ทราบในขณะนี้

2.5.4 ความยาวของกุญแจที่ใช้ในการเข้ารหัส

ความยาวของกุญแจเข้ารหัสมีหน่วยนับเป็นบิต หนึ่งบิตในคอมพิวเตอร์เป็นตัวเลขฐานสองที่ประกอบด้วยค่า 0 และ 1 กุญแจที่มีความยาว 1 บิต ตัวเลขที่เป็นไปได้เพื่อแทนกุญแจนั้น จึงอาจมีค่าเป็น 0 หรือ 1 กุญแจที่มีความยาว 2 บิต ตัวเลขที่เป็นไปได้จึงเป็น 0, 1, 2 และ 3 ตามลำดับ กุญแจที่มีความยาว 3 บิต ตัวเลขที่เป็นไปได้จะอยู่ระหว่าง 0 ถึง 7 ดังนั้นเมื่อเพิ่มความยาวของกุญแจทุกๆ 1 บิต ค่าที่เป็นไปได้ของกุญแจจะเพิ่มขึ้นเป็นสองเท่าตัว หรือจำนวนกุญแจที่เป็นไปได้จะเพิ่มขึ้นเป็น 2 เท่าตัวนั่นเอง

ฉะนั้นจะเห็นได้ว่ากุญแจยิ่งมีความยาวมาก โอกาสที่ผู้บุกรุกจะสามารถคาดเดากุญแจที่ตรงกับหมายเลขที่ถูกต้องของกุญแจจะยิ่งยากมากขึ้นตามลำดับ ในการที่ผู้บุกรุกลองผิดลองถูกกับกุญแจโดยใช้กุญแจที่มีหมายเลขต่างๆ กัน เพื่อหวังที่จะพบกุญแจที่ถูกต้องและสามารถใช้ถอดรหัสข้อมูลได้ การลองผิดลองถูกนี้เราเรียกกันว่าหรือการค้นหากุญแจ (Key search) นั่นเอง ทฤษฎีได้กล่าวไว้ว่าการลองผิดลองถูกนี้โดยเฉลี่ยจะต้องทดลองกับกุญแจเป็นจำนวนครึ่งหนึ่งของกุญแจทั้งหมดก่อนที่จะพบกุญแจที่ถูกต้อง

ความยาวของกุญแจที่มีขนาดเหมาะสมจึงขึ้นอยู่กับความเร็วในการค้นหากุญแจของผู้บุกรุกและระยะเวลาที่ต้องการให้ข้อมูลมีความปลอดภัย ตัวอย่างเช่น ถ้าผู้บุกรุกสามารถลองผิดลองถูก

กับกุญแจเป็นจำนวน 10 กุญแจภายในหนึ่งวินาทีแล้ว กุญแจที่มีความยาว 40 บิต จะสามารถป้องกันข้อมูลไว้ได้ 3,484 ปี ถ้าผู้บุกรุกสามารถลองได้เป็นจำนวน 1 ล้านกุญแจในหนึ่งวินาที (เทคโนโลยีปัจจุบันสามารถทำได้) กุญแจที่มีความยาว 40 บิตจะสามารถป้องกันข้อมูลไว้ได้เพียง 13 วันเท่านั้น (ซึ่งอาจไม่เพียงพอสำหรับในบางลักษณะงาน) ด้วยเทคโนโลยีในปัจจุบันหากผู้บุกรุกสามารถทดลองได้เป็นจำนวน 1,000 ล้านกุญแจในหนึ่งวินาที กุญแจขนาด 128 บิตจะสามารถป้องกันข้อมูลไว้ได้ 1022 ปี ดังนั้นด้วยลักษณะงานทั่วไปกุญแจขนาด 128 บิตจะพอเพียงต่อการรักษาความลับของข้อมูลเอาไว้ได้

ตัวอย่างอัลกอริทึมที่รู้จักกันดี

อัลกอริทึม DES

DES ย่อมาจาก Data Encryption Standard อัลกอริทึมนี้ได้รับการรับรองโดยรัฐบาลสหรัฐอเมริกาในปี ค.ศ. 1977 ให้เป็นมาตรฐานการเข้ารหัสข้อมูลสำหรับหน่วยงานของรัฐทั้งหมด ในปี 1981 อัลกอริทึมยังได้รับการกำหนดให้เป็นมาตรฐานการเข้ารหัสข้อมูลในระดับนานาชาติ ตามมาตรฐาน ANSI (American National Standards) อีกด้วย DES เป็นอัลกอริทึมแบบบล็อกซึ่งใช้กุญแจที่มีขนาดความยาว 56 บิตและเป็นอัลกอริทึมที่มีความแข็งแกร่ง แต่เนื่องด้วยขนาดความยาวของกุญแจที่มีขนาดเพียง 56 บิต ซึ่งในปัจจุบันถือว่าสั้นเกินไป ผู้บุกรุกอาจใช้วิธีการลองผิดลองถูกเพื่อค้นหากุญแจที่ถูกต้องสำหรับการถอดรหัสได้

ในปี 1998 ได้มีการสร้างเครื่องคอมพิวเตอร์พิเศษขึ้นมาซึ่งมีมูลค่า 250,000 เหรียญสหรัฐ เพื่อใช้ในการค้นหากุญแจที่ถูกต้องของการเข้ารหัสข้อมูลหนึ่งๆ ด้วย DES และพบว่าเครื่องคอมพิวเตอร์นี้สามารถค้นหากุญแจที่ถูกต้องได้ภายในระยะเวลาไม่ถึงหนึ่งวัน

อัลกอริทึม Triple-DES

Triple-DES เป็นอัลกอริทึมที่เสริมความปลอดภัยของ DES ให้มีความแข็งแกร่งมากขึ้นโดยใช้อัลกอริทึม DES เป็นจำนวนสามครั้งเพื่อทำการเข้ารหัส แต่แต่ละครั้งจะใช้กุญแจในการเข้ารหัสที่แตกต่างกัน ดังนั้นจึงเปรียบเสมือนการใช้กุญแจเข้ารหัสที่มีความยาวเท่ากับ $56 \times 3 = 168$ บิต Triple-DES ได้ถูกใช้งานกับสถาบันทางการเงินอย่างแพร่หลาย รวมทั้งใช้งานกับโปรแกรม Secure Shell (ssh) ด้วย

อัลกอริทึม Blowfish

Blowfish เป็นอัลกอริทึมที่มีความรวดเร็วในการทำงาน มีขนาดเล็กกระชับ และใช้การเข้ารหัสแบบบล็อก ผู้พัฒนาคือ Bruce Schneier อัลกอริทึมสามารถใช้กุญแจที่มีขนาดความยาวตั้งแต่ไม่มากนักไปจนถึงขนาด 448 บิต ซึ่งทำให้เกิดความยืดหยุ่นสูงในการเลือกใช้กุญแจ รวมทั้งอัลกอริทึมยังได้รับการออกแบบมาให้ทำงานอย่างเหมาะสมกับหน่วยประมวลผลขนาด 32 หรือ 64

บิต Blowfish ได้เปิดเผยสู่สาธารณะและไม่ได้มีการจดสิทธิบัตรใดๆ นอกจากนั้นยังใช้ในโปรแกรม SSH และอื่นๆ

อัลกอริทึม IDEA

IDEA ย่อมาจาก International Data Encryption Algorithm อัลกอริทึมนี้ได้รับการพัฒนาในประเทศสวิตเซอร์แลนด์ที่เมือง Zurich โดย James L. Massey และ Xuejia Lai และได้รับการตีพิมพ์เผยแพร่ในปี ค.ศ. 1990 อัลกอริทึมใช้กุญแจที่มีขนาด 128 บิต และได้รับการใช้งานกับโปรแกรมยอดฮิตสำหรับการเข้ารหัสและลงลายมือชื่ออิเล็กทรอนิกส์ในระบบอีเมลที่มีชื่อว่า PGP ต่อมา IDEA ได้รับการจดสิทธิบัตรทางด้านซอฟต์แวร์โดยบริษัท Ascom-Tech AG ในประเทศสวิตเซอร์แลนด์ ซึ่งทำให้การนำไปใช้ในงานต่างๆ เริ่มลดลง ทั้งนี้เนื่องจากติดปัญหาเรื่องลิขสิทธิ์นั่นเอง

อัลกอริทึม RC4

อัลกอริทึมนี้เป็นอัลกอริทึมแบบสตรีม (ทำงานกับข้อมูลที่ละไบต์) ซึ่งได้รับการพัฒนาขึ้นมาโดย Ronald Rivest และถูกเก็บเป็นความลับทางการค้าโดยบริษัท RSA Data Security ในภายหลังอัลกอริทึมนี้ได้รับการเปิดเผยใน Usenet เมื่อปี ค.ศ. 1994 และเป็นที่ทราบกันว่าเป็นอัลกอริทึมที่มีความแข็งแกร่งโดยสามารถใช้งานความยาวของกุญแจที่มีขนาดตั้งแต่ 1 บิตไปจนกระทั่งถึงขนาด 2048 บิต

อัลกอริทึม Rijndael (หรืออัลกอริทึม AES)

อัลกอริทึมนี้ได้รับการพัฒนาโดย Joan Daemen และ Vincent Rijmen ในปี 2000 อัลกอริทึมได้รับการคัดเลือกโดยหน่วยงาน National Institute of Standard and Technology (NIST) ของสหรัฐอเมริกาให้เป็นมาตรฐานในการเข้ารหัสชั้นสูงของประเทศ อัลกอริทึมมีความเร็วสูงและมีขนาดกะทัดรัดโดยสามารถใช้กุญแจที่มีความยาวขนาด 128, 192 และ 256 บิต

อัลกอริทึม One-time Pads

อัลกอริทึมนี้ได้รับการยอมรับว่าเป็นอัลกอริทึมที่ไม่มีใครสามารถเจาะความแข็งแกร่งของอัลกอริทึมได้ อัลกอริทึมใช้กุญแจที่มีความยาวซึ่งอาจจะมากกว่าขนาดความยาวของข้อความที่ต้องการเข้ารหัส กุญแจจะถูกสร้างออกมาแบบสุ่มและโดยปกติจะถูกใช้งานแค่เพียงครั้งเดียวแล้วทิ้งไป แต่ละไบต์ของข้อความที่ต้องการส่งไปจะถูกเข้ารหัสและถอดรหัสโดยหนึ่งไบต์ (ชนิดไบต์ต่อไบต์) ของกุญแจที่ถูกสร้างขึ้นมาใช้งาน เนื่องจากกุญแจที่ถูกใช้งานแต่ละครั้งจะไม่ซ้ำกันและถูกสร้างขึ้นแบบสุ่ม จึงเป็นการยากที่จะค้นหากุญแจที่ถูกต้องได้

ข้อจำกัดของอัลกอริทึมนี้ คือขนาดของกุญแจที่อาจมีขนาดยาวกว่าข้อความที่ต้องการส่ง ซึ่งส่งผลให้การส่งมอบกุญแจที่มีขนาดใหญ่ทำได้ไม่สะดวกนัก รวมทั้งการสร้างกุญแจให้มีความสุ่มสูงไม่ใช่เป็นสิ่งที่ทำได้ง่ายนัก อย่างไรก็ตามอัลกอริทึมนี้ก็ยังมีการใช้งานในระบบเครือข่ายที่ต้องการความปลอดภัยสูง

อัลกอริทึมสำหรับการเข้ารหัสแบบกุญแจสาธารณะ (หรือการเข้ารหัสแบบอสมมาตร) อัลกอริทึมที่เป็นที่รู้จักกันดีมีดังนี้

อัลกอริทึม RSA

อัลกอริทึม RSA ได้รับการพัฒนาขึ้นที่มหาวิทยาลัย MIT ในปี 1977 โดยศาสตราจารย์ 3 คน ซึ่งประกอบด้วย Ronald Rivest, Adi Shamir และ Leonard Adleman ชื่อของอัลกอริทึมได้รับการตั้งชื่อตามตัวอักษรตัวแรกของนามสกุลของศาสตราจารย์ทั้งสามคน อัลกอริทึมนี้สามารถใช้ในการเข้ารหัสข้อมูลรวมทั้งการลงลายมือชื่ออิเล็กทรอนิกส์ด้วย

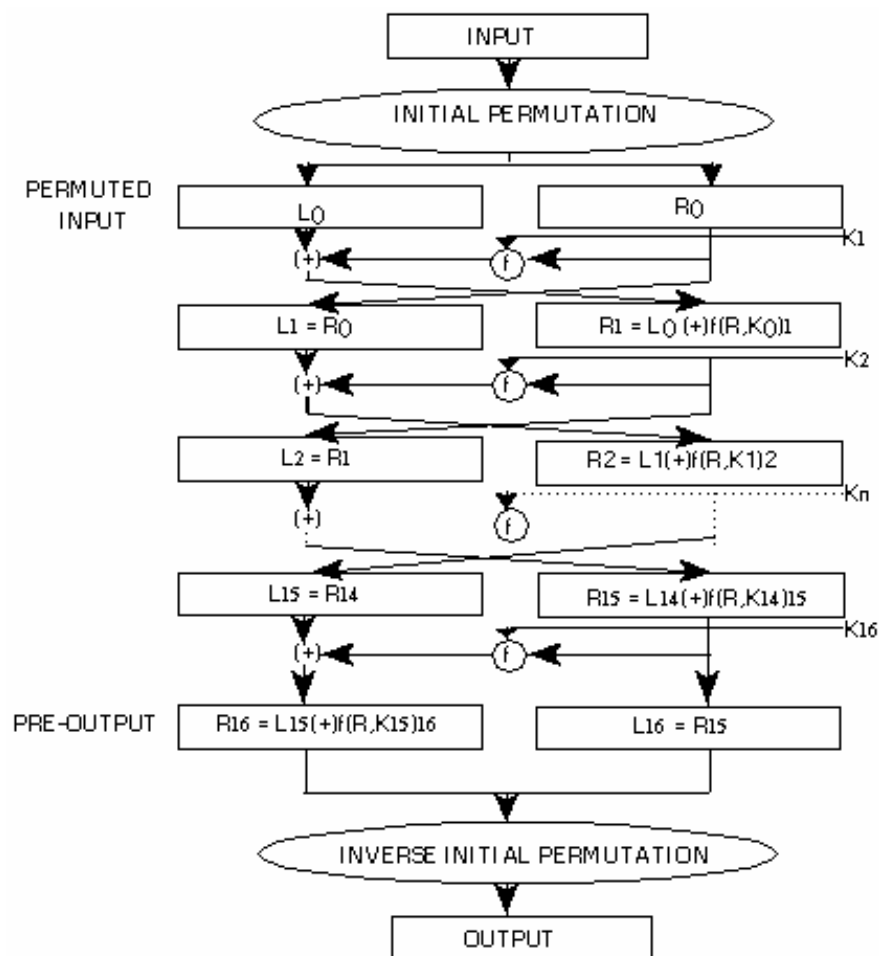
อัลกอริทึม DSS

DSS ย่อมาจาก Digital Signature Standard อัลกอริทึมนี้ได้รับการพัฒนาขึ้นมาโดย National Security Agency ในประเทศสหรัฐอเมริกาและได้รับการรับรองโดย NIST ให้เป็นมาตรฐานกลางสำหรับการลงลายมือชื่ออิเล็กทรอนิกส์ในประเทศสหรัฐอเมริกา

2.6 การเข้ารหัสลับแบบ DES

การทำงานของอัลกอริทึมแบบ DES นั้นเป็นพื้นฐานของการทำงานแบบ 3DES ดังนั้นในเบื้องต้นต้องเข้าใจการทำงานของ DES ซึ่งเป็นการเข้ารหัสลับที่เป็นแบบสมมาตร ใช้ขนาดข้อมูล 64 บิต และขนาดของกุญแจ 56 บิต การทำงานของอัลกอริทึม DES ได้แบ่งการทำงานออกเป็น 3 ส่วนที่สำคัญ คือ

1. การเรียงบิตใหม่ (Initial Permutation) เพื่อผลิต “permuted input” ซึ่งเป็นข้อมูลที่มีการสลับตำแหน่งจากเดิมไป
2. การเข้าฟังก์ชันการคำนวณ โดยมีกระบวนการทำงานแบบเดิมซ้ำกัน 16 ครั้ง ซึ่งผลลัพธ์ที่ได้จะได้เป็นข้อมูลขนาด 64 บิตอยู่ชุดหนึ่งที่เรียกว่า PREOUTPUT
3. การเรียงบิตกลับ (Inverse Initial Permutation) ซึ่งจะได้ผลลัพธ์ออกมาเป็นข้อมูลที่เข้ารหัสแล้ว



รูปที่ 2.9 กระบวนการเข้ารหัสด้วยอัลกอริทึม DES

สำหรับกระบวนการถอดรหัส นั้น ทำเช่นเดียวกันกับกระบวนการเข้ารหัส คือ ใส่ข้อมูลที่ถูกเข้ารหัสข้อมูลมาแล้วที่มีขนาด 64 บิต พร้อมกับกุญแจตัวเดิมที่ใช้ในการเข้ารหัส แล้วทำตามขั้นตอนต่างๆ เหมือนกับการเข้ารหัสข้อมูล แต่ต้องทำการเปลี่ยนลำดับของกุญแจใหม่ โดยใช้กุญแจจากลำดับท้ายสุด เรียงมาจนถึงลำดับแรกสุด คือ $K_{16}, K_{15}, \dots, K_1$

2.7 การเข้ารหัสแบบ 3DES (Triple Data Encryption Standard)

การเข้ารหัสแบบ 3DES นั้น เป็นการเข้ารหัสอีกแบบหนึ่ง โดยอิงมาตรฐานมาจากการเข้ารหัสแบบ DES ซึ่งเข้าและถอดรหัสแบบ DES เป็นจำนวน 3 ครั้ง และใช้กุญแจเป็นจำนวน 3 ตัวในการทำงาน ดังนั้นเปรียบเสมือนว่าใช้กุญแจที่มีความยาว 56×3 คือ 168 บิต

TRIPLE DES BLOCK DIAGRAM (ECB Mode)

TDEA Encryption Operation:

$$I \rightarrow \boxed{\text{DES } E_{K1}} \rightarrow \boxed{\text{DES } D_{K2}} \rightarrow \boxed{\text{DES } E_{K3}} \rightarrow O$$

TDEA Decryption Operation:

$$I \rightarrow \boxed{\text{DES } D_{K3}} \rightarrow \boxed{\text{DES } E_{K2}} \rightarrow \boxed{\text{DES } D_{K1}} \rightarrow O$$

รูปที่ 2.10 กระบวนการเข้ารหัสและถอดรหัสด้วย 3DES

โดยมีขั้นตอนในการเข้ารหัสข้อมูล ดังนี้คือ

1. เข้ารหัสข้อมูลโดยใช้กุญแจตัวแรก
2. ถอดรหัสข้อมูลโดยใช้กุญแจตัวที่สอง และ
3. เข้ารหัสข้อมูลโดยใช้กุญแจตัวที่สาม

ในส่วนของขั้นตอนของการถอดรหัสข้อมูล มีดังนี้คือ

1. ถอดรหัสข้อมูลโดยใช้กุญแจตัวแรก
2. เข้ารหัสข้อมูลโดยใช้กุญแจตัวที่สอง และ
3. ถอดรหัสข้อมูลโดยใช้กุญแจตัวที่สาม

2.8 Mode of operation

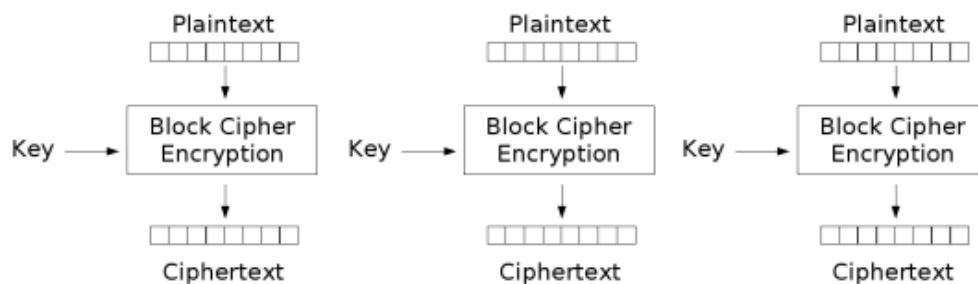
การเข้ารหัสและถอดรหัสลับนั้นอัลกอริธึมที่ใช้จะมองข้อความที่จะเข้ารหัสและถอดรหัสเป็นบล็อก ซึ่งความยาวของบล็อกมักเป็นค่าคงที่ขนาด 64 หรือ 128 บิต ดังนั้นหากข้อความที่ต้องการเข้ารหัสลับนั้นมีค่าความยาวมากกว่านี้ก็ต้องใช้ Mode of operation เข้ามาช่วยแบ่งข้อมูลที่ยาว ๆ นั้นให้เป็นหลายๆบล็อก ซึ่งจะมีหลาย operation ดังนี้

1. Electronic Codebook(ECB)
2. Cipher Block Chaining (CBC)
3. k-Bit Output Feedback Mode (OFB)
4. k-Bit Cipher Feedback Mode (CFB)
5. Counter Mode (CTR)

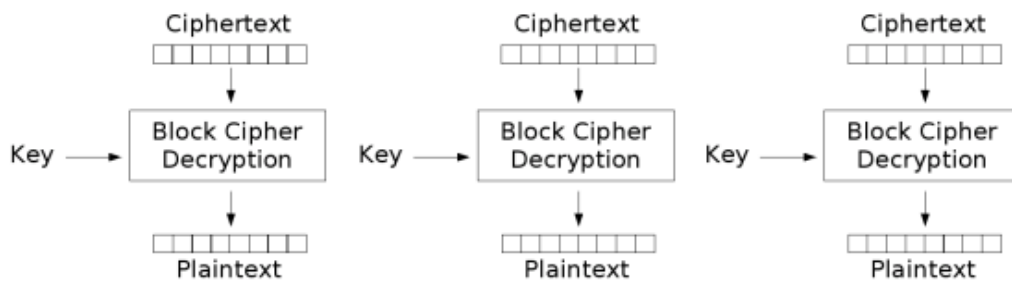
ขั้นตอนการทำงานของแต่ละวิธี

1. Electronic Codebook (ECB)

เป็นวิธีการที่ง่ายที่สุด โดยหลักการทำงานคือ นำเอาข้อความ(plaintext) มาแบ่งเป็นบล็อก เช่นขนาดบล็อกละ 64 บิตถ้าเหลือก็จะทำการเติมเต็มให้ครบ 64 บิต แล้วนำมาเข้ารหัสลับกับกุญแจลับที่เลือกไว้ทีละบล็อกโดยทุกบล็อกนั้นใช้กุญแจตัวเดียวกัน และในการถอดรหัสลับก็ทำในลักษณะเดียวกัน คือนำไซเฟอร์(cipher)ที่ได้จากการเข้ารหัสมาถอดรหัสด้วยกุญแจตัวเดิมซึ่งเหมือนทุกบล็อก ลักษณะการทำงานเป็นดังรูปที่ 2.11 แสดงไว้ข้างล่างนี้



Electronic Codebook (ECB) mode encryption



Electronic Codebook (ECB) mode decryption

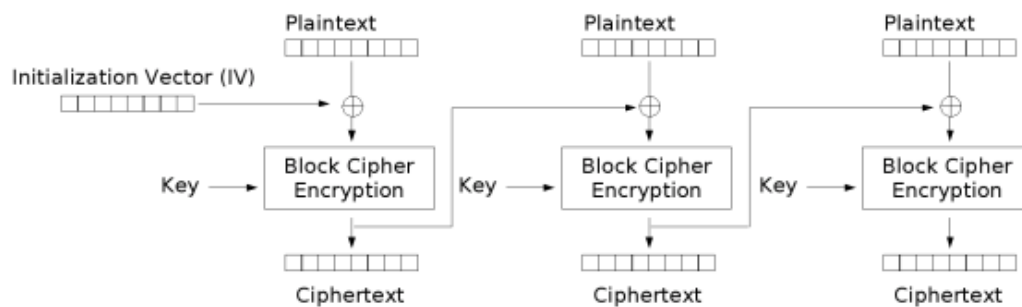
รูปที่ 2.11 แสดงการเข้ารหัสและถอดรหัสลับแบบ ECB

ข้อเสียของวิธีการนี้มีอยู่ 2 ประการคือ

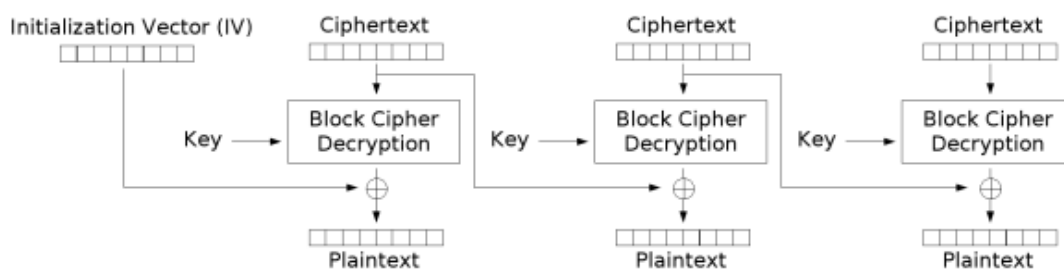
1. เนื่องจากทุกบล็อกนั้นใช้กุญแจเดิม ดังนั้นหากข้อความเดิมที่เหมือนเดิมก็จะทำให้ได้ไซเฟอร์ออกมาเหมือนกันทำให้สามารถนำไปสู่การคาดเดาที่ง่ายขึ้นได้
2. ปัญหาด้านการตรวจสอบความถูกต้อง เพราะหากทำการสลับไซเฟอร์กันก็ไม่สามารถทำได้เพราะแต่ละบล็อกไม่ได้มีความต่อเนื่องกัน

2. Cipher Block Chaining (CBC)

เป็นการนำ ECB มาพัฒนา แก้ไขในจุดที่ยังเป็นปัญหาของ ECB โดยมีการนำ initial vector เข้ามาช่วย ป่วนให้เกิดความวุ่นวายมากขึ้นซึ่งเป็นค่าที่ได้จากการสุ่ม โดย IV ที่ได้นั้นเอามา XOR กับบล็อกข้อความ แล้วทำการเข้ารหัสลับเพื่อให้ได้เป็นไซเฟอร์ออกมาแล้วเอาไซเฟอร์นั้นไปเป็น IV สำหรับข้อความบล็อกต่อไปตามลำดับ



Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

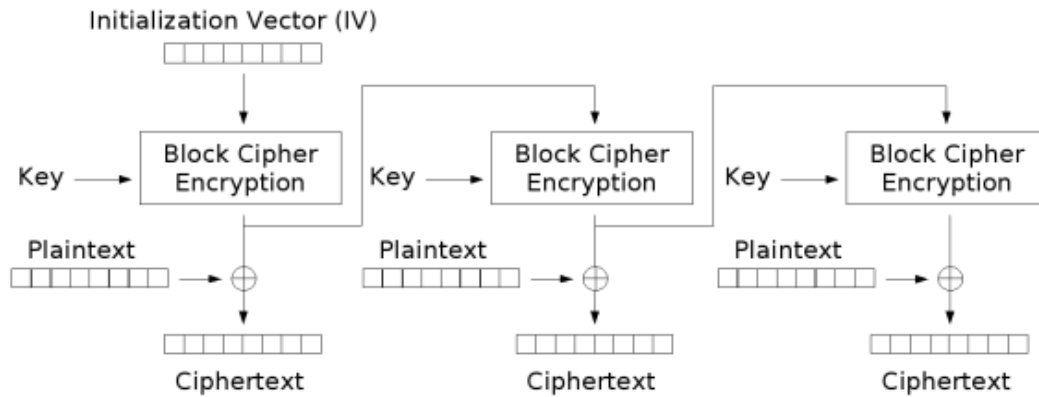
รูปที่ 2.12 แสดงการเข้ารหัสและถอดรหัสลับแบบ CBC

ข้อดีของวิธีการนี้คือ แม้ว่าข้อความจะเป็นข้อความเดิมแต่จะได้ Cipher ที่ไม่เหมือนเดิม เพราะว่าใช้ค่า IV ที่ต่างไปจากเดิมและแก้ไขการสลับบล็อกได้ในระดับหนึ่งเพราะหากบล็อกเปลี่ยนที่ค่าที่ได้จากการถอดรหัสออกมาจะเปลี่ยนไป

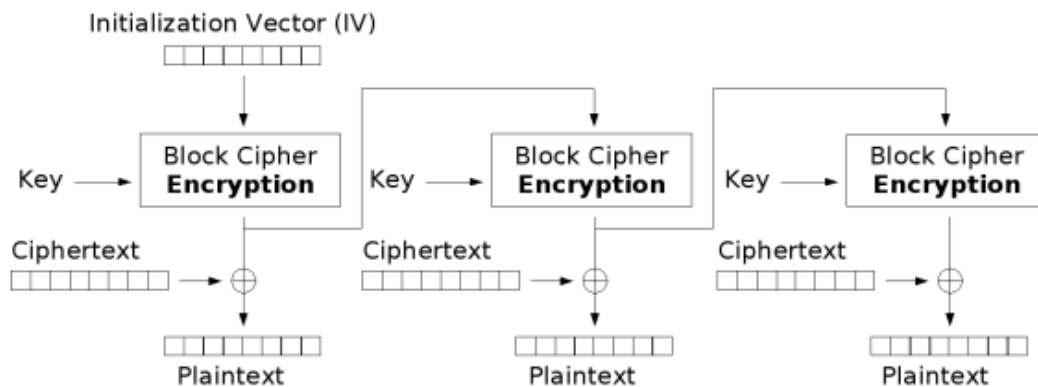
ข้อเสียของวิธีการนี้คือ เปลืองแบนด์วิธ เพราะเวลาส่งต้องส่งทั้ง Cipher และ IV ไปด้วย และยังสามารถทำการเปลี่ยนแปลงบิตได้อยู่ดีทำให้ความหมายที่ออกมาผิดไป

3. k-Bit Output Feedback Mode (OFB)

เป็น Stream cipher ที่ XOR message แบบ one time padding ทำให้เกิดบล็อกข้อมูลขนาดไหนก็ได้ไม่จำเป็นต้องเป็น 64 หรือ 128 บิต ทำให้ไม่จำเป็นต้องมีการเติมเต็มบิตหากข้อมูลไม่เต็มบล็อก



Output Feedback (OFB) mode encryption



Output Feedback (OFB) mode decryption

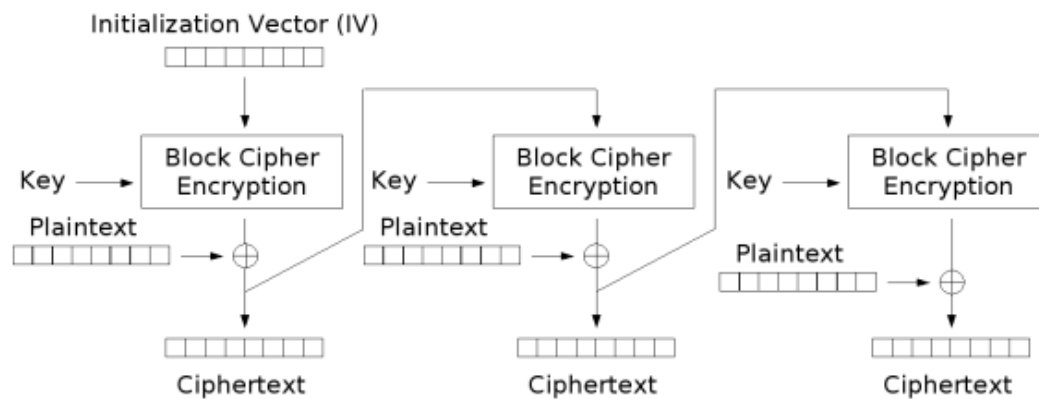
รูปที่ 2.13 แสดงการเข้ารหัสและถอดรหัสลับแบบ OFB

ข้อดี

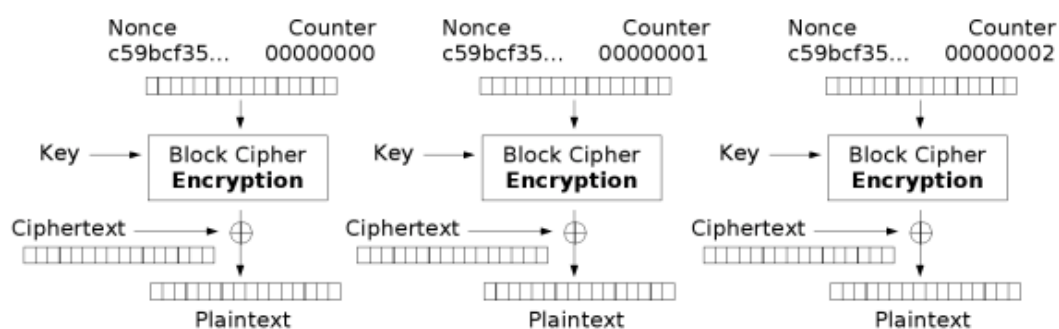
1. สามารถหา subkey ไว้ก่อนได้ทำให้สามารถทำได้เร็ว
2. ข้อมูลไม่จำเป็นต้องเต็มบล็อก ทำให้ไม่ต้องทำการเติมเต็มบิตข้อมูล

4. k-Bit Cipher Feedback Mode (CFB)

คล้ายกับการทำงานของ OFB แต่ค่าที่ใช้ในการเข้ารหัสลับกับบล็อกต่อไปนั้นใช้ค่าที่ได้จากการเข้ารหัสลับของบล็อกที่ผ่านมา



Cipher Feedback (CFB) mode encryption



Counter (CTR) mode decryption

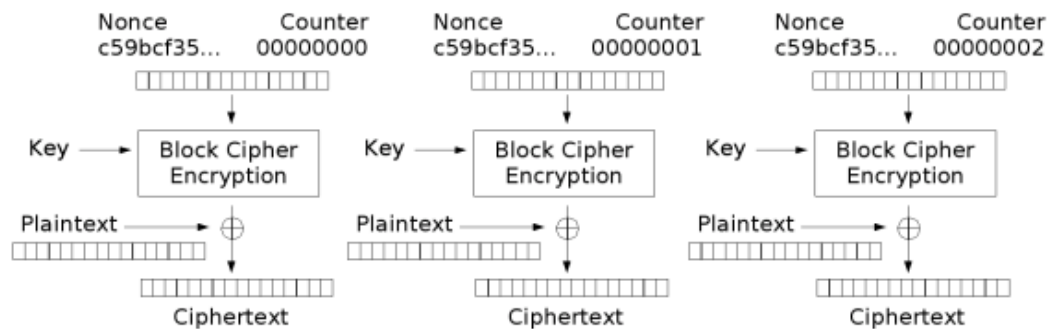
รูปที่ 2.14 แสดงการเข้ารหัสและถอดรหัสลับแบบ CFB

ข้อดีของวิธีการนี้คือ ไม่สามารถเปลี่ยนแปลง Cipher ได้เพราะ C_n มีผลต่อการคิดในบล็อกต่อไป

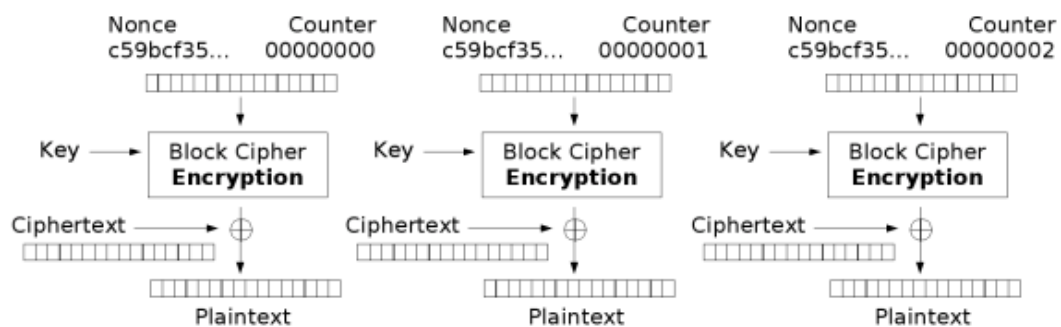
ข้อเสียของวิธีการนี้คือ ซับซ้อนและยุ่งยาก ไม่สามารถทำงานแบบขนาน(Parallel) ไม่สามารถทำการสร้างกุญแจ (generate key) ไว้วางหน้าได้

5. Counter Mode (CTR)

คล้ายกับการทำงานของ feedback mode แต่จะเปลี่ยนค่าตรง register ไปเป็นค่าเวกเตอร์ตั้งต้น (initial vector) แทน ข้อดีของวิธีการนี้คือ สามารถแยกงานทำได้คือ สามารถใช้ thread แยกงานกันทำบล็อกไหนก็ได้ ถ้าทำการสลับบล็อกไซเฟอร์แล้วจะทำให้การถอดรหัสนั้นมีปัญหา เพราะว่าค่าของเวกเตอร์ตั้งต้นเป็นตัวบังคับลำดับของบล็อก



Counter (CTR) mode encryption



Counter (CTR) mode decryption

รูปที่ 2.15 แสดงการเข้ารหัสและถอดรหัสลับแบบ CTR

สำหรับการเข้ารหัสและถอดรหัสลับแบบ 3DES นั้น จะใช้ CBC ซึ่งสามารถทำได้ 2 แบบ คือ CBC Outside และ CBC Inside โดยสองแบบนี้จะมีลักษณะที่แตกต่างกันคือ

ตารางที่ 2.1 เปรียบเทียบคุณสมบัติของ CBC inside และ CBC outside

CBC inside	CBC outside
เป็นวิธีการที่มีความซับซ้อนมาก	เป็นที่นิยมมากกว่าเพราะว่ามองแยกเป็นส่วนๆ
ดีกว่า outside ในเชิงด้านความปลอดภัย	มีความสะดวกในการทำงานมากกว่า

2.9 อัลกอริธึม AES

การเข้ารหัสแบบ AES (Advanced Encryption Standard)

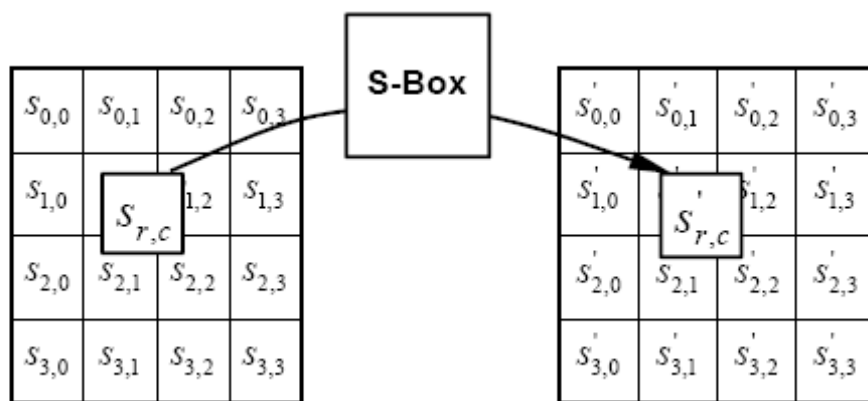
AES หรือที่รู้จักว่า Rijndael คือการเข้ารหัสข้อมูลโดยจะแบ่งขนาดบล็อกเป็นบล็อกละ 128 บิต และมีการใช้กุญแจขนาด 128, 192 หรือ 256 บิตแต่ถ้าตรงตามมาตรฐานจะใช้คีย์ขนาด 128 บิต ด้วย ซึ่งการเข้ารหัสและถอดรหัสนั้นเป็นการเข้ารหัสแบบสมมาตร (Symmetric Cryptosystem) คือ

ใช้กุญแจตัวเดียวกันในการเข้าและถอดรหัส โดยจะมองเป็นเมตริก 2 มิติขนาด 4×4 โดยบล็อกละ 8 บิต โดยทำงานตามสี่ขั้นตอนนี้ให้ครบ 10 รอบ

การทำงานของอัลกอริทึม AES แบ่งเป็น 2 ส่วนคือ ส่วนของข้อความและส่วนของกุญแจที่ใช้

ลำดับการทำงานของส่วนข้อความแบ่งออกเป็น 4 ขั้นตอนที่สำคัญคือ

1. การเทียบเปลี่ยนค่า (SubBytes) นำเอาข้อความที่ต้องการนำมาเข้ารหัสลับ มาแบ่งเป็น บล็อกขนาดบล็อกละ 8 บิตทั้งหมดได้ความยาว 128 บิต แล้วนำเอาค่าที่อยู่ในข้อความตั้งต้นเทียบค่า กับตาราง s-box แล้วได้เป็นค่าใหม่ออกมา

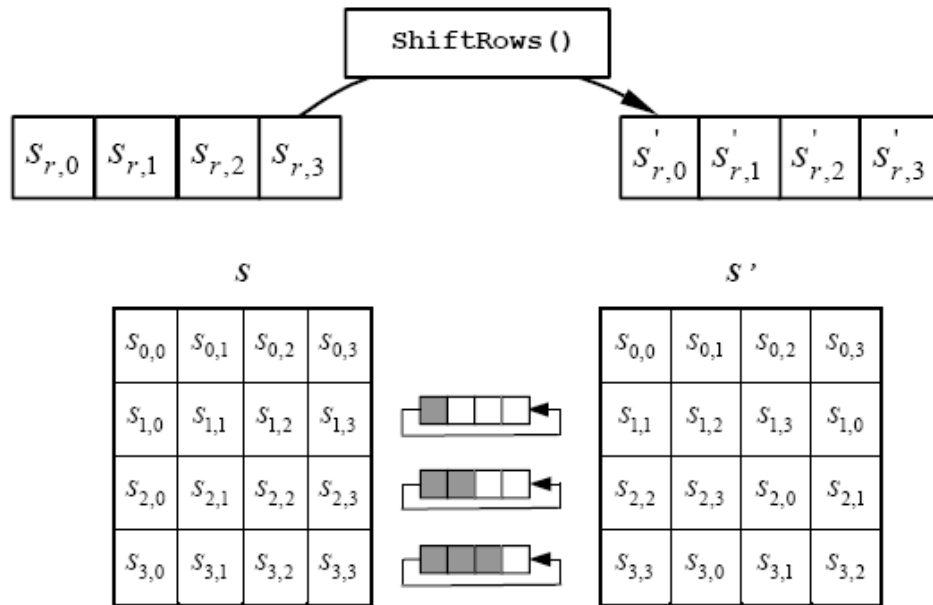


รูปที่ 2.16 แสดงค่าที่ได้ออกมาโดยใช้ s-box ในการเทียบแต่ละไบต์

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

รูปที่ 2.17 แสดง s-box ได้ค่าโดยเทียบไบต์ xy (แสดงออกมาในรูปแบบเลขฐาน 16)

2. การเลื่อนแถว (ShiftRows) เป็นการเปลี่ยนแปลงข้อความที่ถูกเข้ารหัสโดยการเลื่อน 3 แถวล่างเนื่องจากมี 4 แถว เริ่มจาก 0-3 เริ่มเลื่อนแถวที่ 1 เลื่อนซ้ายไป 1 บล็อก แถวที่ 2 เลื่อนไป 2 บล็อก แถวที่ 3 เลื่อนไป 3 บล็อก เป็นการเลื่อนวนไปทางซ้ายดังรูป



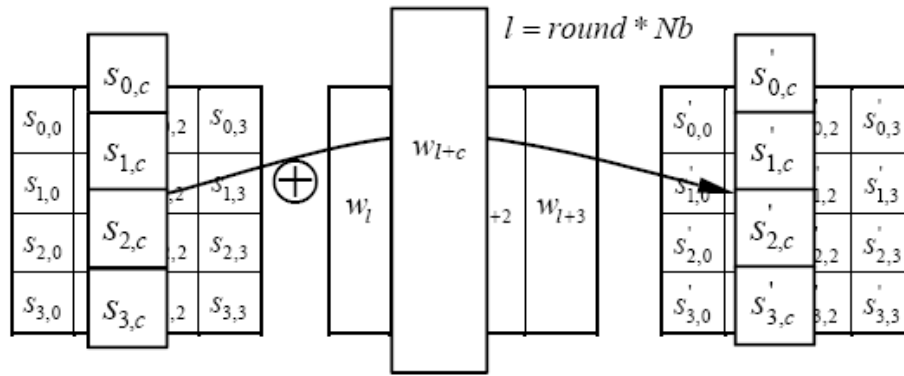
รูปที่ 2.18 แสดงการเลื่อนแถวซ้ายที่ 3 แถวล่าง

3. การผสมคอลัมน์ (MixColumns) เป็นการคิดเทียบต่อคอลัมน์โดยคิดแบบโพลิโนเมียลผ่านฟังก์ชัน GF^8 ซึ่งจะแสดงเป็นการคำนวณทางเมตริกซ์ตามรูปข้างล่างนี้ เพื่อให้ค่าที่ออกมาได้จากการผสมแถวให้มีค่าเปลี่ยนไป

$$\begin{bmatrix} S'_{0,c} \\ S'_{1,c} \\ S'_{2,c} \\ S'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} S_{0,c} \\ S_{1,c} \\ S_{2,c} \\ S_{3,c} \end{bmatrix} \quad \text{สำหรับ } 0 \leq c \leq 4$$

รูปที่ 2.19 แสดงการผสมคอลัมน์

4. การนำคีย์มาผสมกับข้อความ (AddRoundKey) เป็นการ XOR แต่ละบิตกับคีย์ที่ได้จากการคำนวณในแต่ละรอบ



รูปที่ 2.20 แสดงผลลัพธ์ที่เกิดจากการ XOR กุญแจกับค่าแต่ละบิต